Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

SmartFile: C++ Based File Compression and Decompression Using Huffman Coding

Team Name: *Team #*	*BitMasters*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Negi , Rishi - 2510380076* *2510380076@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bansal , Vardan - 2510372517* *2510372517@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Rautela, Priyansh - 2510370501* *2510370501@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Saksham , Pragya - 2510370412* *2510370412@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

SmartFile is a C++ project developed to reduce file size using Huffman Coding. The main objective is to save storage space without losing the original data. The compressed file should be restored to its original form during decompression.

The program reads files in binary mode and counts the frequency of each byte. These frequencies are used to build a Huffman Tree with a priority queue. The tree is then used to assign binary codes to the bytes. The encoded data is written using bit-level processing instead of storing each code as ordinary text.

The project uses a custom .sfile format to store compressed data and the information required for decompression. It also includes a RAW storage mode for cases where compression may increase the file size because of additional metadata.

The initial implementation includes the basic program structure, byte-frequency counting, Huffman Tree construction, code generation and BitWriter. Work is still required on BitReader, decompression, metadata handling and complete testing.

Updated Project Approach and Architecture

SmartFile follows a sequence of steps to process a file. The program opens the selected input file in binary mode and reads it byte by byte. This allows the same process to work with different types of files.
Create a frequency table showing how many times each byte occurs.The Huffman Tree is made by using a Priority Queue, which works like a min-heap.
Once the tree is built, the program assigns a binary code (0s and 1s) to each byte by goingthrough the tree.The BitWriter and BitReader classes help the program write and read data bit by bit, so the encoded data can be stored as bits instead of regular characters.
The program saves the file in a custom .sfile format. Its header keeps important information needed to decompress the file, like the original file size and the frequency of each byte.
During decompression, the program reads the saved information from the header and uses the byte frequency data to build the Huffman Tree again.The program reads the encoded bits and decodes them using the rebuilt Huffman Tree. It then writes the decoded bytes to make the original file again.
The program uses OOP to keep the code simple and organized. Different classes like HuffmanNode, BitWriter, BitReader, and Compressor are used for different parts of the program.After compression or decompression, the program shows the original file size, the new file size, and how much space was saved.
The program is being developed using standard C++ libraries, including <iostream>, <fstream>, <queue>, <string> and <filesystem>. It runs locally on the user's computer and does not require network communication.

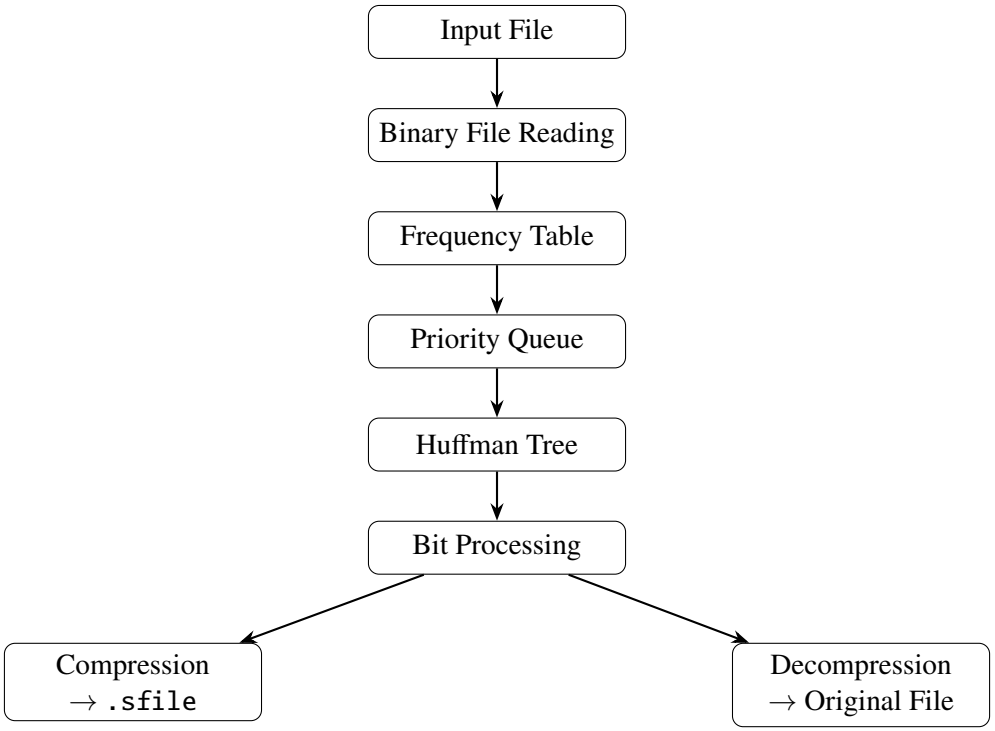


Figure 1: Architecture of the SmartFile Compression and Decompression System

Tasks Completed

Task Completed	Team Member
Requirements analysis, project structure, and Huffman-based system design	Rishi Negi
Frequency table, Huffman Tree construction, min-heap, and Huffman code generation	Vardan Bansal
Initial C++ menu, binary file handling, and BitWriter implementation	Priyansh Rautela
Initial compression testing, output checking, and test-case preparation	Pragya Saksham

Challenges/Roadblocks

The main challenges in the SmartFile project and their solutions are given below.

1. **Binary Data Handling:** Reading and writing files correctly without changing the original data.
2. **Bit-Level Storage:** Storing data as actual bits instead of storing 0 and 1 as normal text characters
3. **Huffman Tree Management:**Creating and managing Huffman Tree nodes using dynamic memory in C++.
4. **Metadata Handling:** Small files may become larger after compression because extra information is added. RAW mode can help solve this problem.
5. **Small File Overhead:** Small files may become larger after compression because extra information is added. RAW mode can help solve this problem.
6. **Size and Percentage Calculation:**Calculating file sizes and compression percentages correctly, especially when the compressed file is larger than the original.
7. **Testing Different Files:** Testing text files, binary files, small files, and already-compressed files to check whether the application works correctly with different types of data.

Tasks Pending

Task Pending	Team Member
Implementation of the `.sfile` format and compression metadata	Rishi Negi
BitReader implementation and file decompression logic	Vardan Bansal
RAW mode, file-size comparison, and compression statistics	Priyansh Rautela
Integrated testing, round-trip verification, and test documentation	Pragya Saksham

Project Outcome/Deliverables

The project aims to develop a file compression and decompression program in C++ using Huffman Coding. The main deliverables are:

1. A C++ program for compressing and decompressing files.
2. Huffman Tree construction based on byte frequencies.
3. Bit-level processing to store encoded data.
4. A custom `.sfile` format for storing compressed data and the information required for decompression.
5. RAW mode for files where compression does not reduce the file size.
6. Display of original file size, compressed file size, space saved, and compression percentage.
7. Testing to check whether decompressed files match the original files.

The final goal is to recover the original file without losing or changing its data.

Progress Overview

About 45% of the planned work has been done so far. The team has discussed the project requirements, prepared the basic design and decided which modules need to be developed.
The initial coding has started. The program can count how often each byte appears in a file using a frequency array. A priority queue is being used for Huffman Tree construction, and Huffman codes have been generated. The `BitWriter` class has also been developed to write encoded data bit by bit.
The complete program is not ready yet. Work is still needed on the `.sfile` format and the `BitReader` class. Compression and decompression modules also need to be connected. RAW mode, compression statistics and error handling require further work and testing.
Next, the team will focus on completing these parts and making the modules work together. Files of different sizes and types will be used for testing. The decompressed output will then be compared with the original file to check whether the data remains unchanged.

Codebase Information

Repository: The SmartFile project files are stored both on the local computer and in a GitHub repository for easy access and code management.The repository is available at: https://github.com/Vardanbansalgeu/pbl$_sem3$
The project includes the initial structure and ongoing development work related to Huffman Coding, file handling, and the compression and decompression workflow.
Branch Status: The team is using the existing GitHub repository for the project. A separate development branch has not been created so far. If needed, the team may create one during the later stages of development.
Version Control and Team Collaboration: Git is being used to keep track of changes in the project files. Some commits have already been made to the repository. Each team member is working on the tasks assigned to them.
The repository keeps the source code in one place and makes it easier to check previous changes. As development continues, the team will commit code updates, bug fixes and changes made during module integration and testing.
At present, the main focus is on finishing the remaining code and connecting the compression and decompression parts. The application will then be tested to find errors and check whether it works as expected.

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Compilation / Menu Flow	PASS	Initial C++ build and menu flow checked.
Small-File Size Handling	PASS	Header overhead observed; RAW fallback used.
Compression Statistics	FAIL	Size-increase percentage handling needs refinement.
Full Round-Trip Validation	PENDING	Complete compression and decompression comparison remains pending.

Deliverables Progress

Completed Tasks

- Created the basic C++ project structure and implemented binary file reading and writing.
- Added input and output file selection through the menu.
- Implemented byte-frequency calculation using an array of 256 positions.
- Created the HuffmanNode structure and min-heap using `priority_queue`.
- Implemented Huffman Tree construction by combining the two lowest-frequency nodes.
- Generated Huffman codes by assigning 0 to the left branch and 1 to the right branch.
- Created the BitWriter class with bit-level writing, buffering, and flushing operations.
- Connected the frequency table, Huffman Tree, and generated codes to encode input data into compressed binary output.

Tasks In Progress

- Designing the custom `.sfile` format and metadata handling.
- Implementing the BitReader class and decompression logic.
- Integrating RAW mode and file-size comparison.
- Developing compression statistics, including bytes saved and space-saving percentage.
- Integrating the modules and improving error handling.

Pending Tasks

- Complete compression and decompression integration.
- Test text files, binary files, and different file sizes.
- Verify that decompressed files match the original files exactly.
- Validate Huffman and RAW modes, including edge cases.
- Organize the final `.h` and `.cpp` files and complete the documentation.
- Prepare the final working demonstration.